

1 Tworzenie automatów z wyrażeń regularnych

1.1 Cel ćwiczenia

Celem ćwiczenia jest ugruntowanie znajomości programów `flex` i `bison`, utrwalenie umiejętności dokonywania analizy składniowej oraz rozszerzenie wiadomości na temat automatów skończonych.

1.2 Środowisko

Składnia polecenia `dot`:

```
dot -Tps < źródło > wynik.ps
```

Wynik najłatwiej uzyskać pisząc np.:

```
echo '0(0|1)*0' | ./z7b | dot -Tps > wynik.ps; gv wynik.ps &
```

1.3 Tworzenie automatów na podstawie wyrażeń regularnych – metoda Champarnaud-Głuszkow

Automat tworzony jest z części składowych w podobny sposób, w jaki oblicza się wartość wyrażenia arytmetycznego. Cechą charakterystyczną automatów Głuszkowa powstających w tej metodzie jest to, że wszystkie przejścia wchodzące do danego stanu mają tę samą etykietę. W wyrażeniu regularnym podstawowymi elementami są: zbiór pusty $\emptyset$, pusty ciąg symboli ϵ , oraz symbol σ z alfabetu Σ .

Pustemu zbiorowi odpowiada automat ze stanem początkowym bez przejść. **Pustemu ciągowi symboli ϵ** odpowiada automat ze stanem początkowym będącym jednocześnie stanem końcowym. **Symbolowi z alfabetu $a \in \Sigma$** odpowiada automat ze stanem początkowym i stanem końcowym oraz z przejściem między tymi stanami etykietowanym tym symbolem.

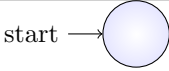
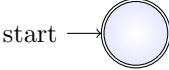
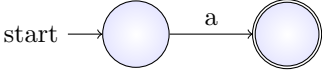
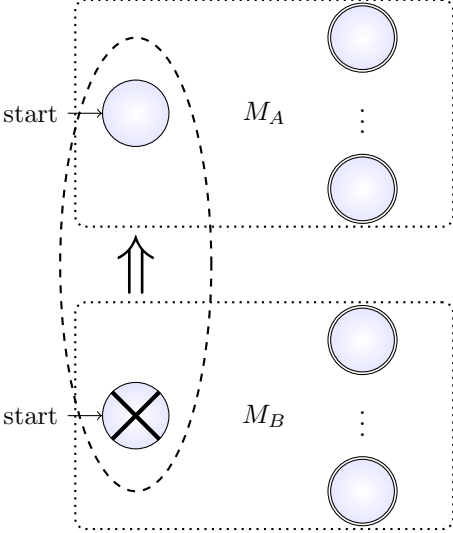
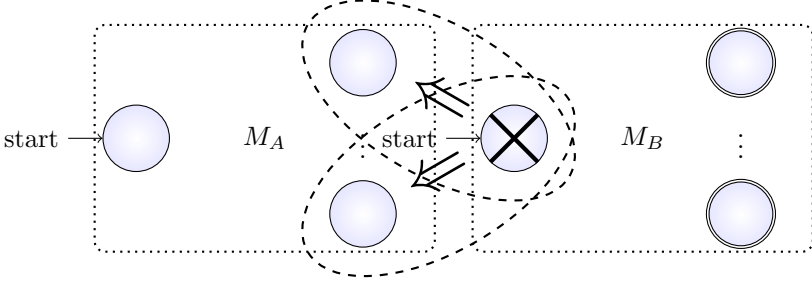
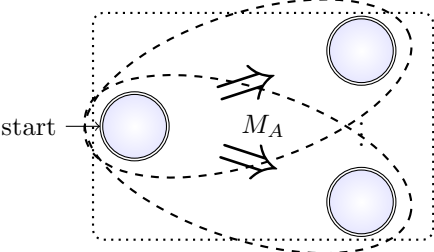
Mając podstawowe cegiełki, można tworzyć większe automaty korzystając ze sklejania wyrażeń, alternatywy i domknięcia przechodniego. Wyrażeniu regularnemu R odpowiada automat $M = (Q, \Sigma, \delta, q_0, F)$, gdzie Q to zbiór stanów, Σ – alfabet, δ – funkcja przejścia, $q_0 \in Q$ – stan początkowy, $F \subseteq Q$ – stany końcowe. Wyrażeniom R_A i R_B odpowiadają automaty $M_A = (Q_A, \Sigma, \delta_A, q_{0A}, F_A)$ i $M_B = (Q_B, \Sigma, \delta_B, q_{0B}, F_B)$. Automat dla **sklejania** $R_A R_B$ wyrażeń R_A i R_B tworzymy dodając wszystkie przejścia stanu początkowego q_{0B} automatu M_B do każdego stanu końcowego automatu M_A . Jeśli q_{0B} jest stanem końcowym, stany końcowe M_A pozostają stanami końcowymi, w przeciwnym przypadku przestają nimi być. Stan q_{0B} zostaje usunięty. Stanem początkowym q_0 automatu wynikowego będzie stan q_{0A} , a stanami końcowymi – stany końcowe automatu M_B oraz, jeżeli q_{0B} jest stanem końcowym, stany końcowe automatu M_B .

Automat dla **alternatywy** $R_A | R_B$ wyrażeń R_A i R_B tworzymy łącząc stany początkowe automatów M_A i M_B w jeden stan, tzn. dodając wszystkie przejścia wychodzące z q_{0B} do stanu q_{0A} i czyniąc q_{0A} stanem końcowym, jeśli stan początkowy M_B był stanem końcowym. Stanem początkowym automatu wynikowego staje się q_{0A} , stanami końcowymi – suma stanów końcowych M_A i M_B (ew. z pominięciem q_{0B} , a z uwzględnieniem q_{0A} , jeśli $q_{0B} \in F_B$).

Automat dla **domknięcia przechodniego** R_A^* wyrażenia R_A tworzymy dodając wszystkie przejścia stanu q_{0A} do stanów końcowych automatu M_A i czyniąc stan q_{0A} stanem końcowym. Stanem początkowym automatu wynikowego staje się q_{0A} , stanami końcowymi – $F_A \cup \{q_{0A}\}$.

W wyrażeniu regularnym można też stosować **nawiasy** do grupowania na podobnym zasadach, jak w wyrażeniach arytmetycznych. Konstrukcję Champarnaud-Głuszkow podsumowuje poniższa tabela, zaś dodatkowe informacje na temat przedstawionych zagadnień można znaleźć w publikacjach:

- John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Wprowadzenie do teorii automatów, języków i obliczeń*, Wydawnictwo Naukowe PWN, Warszawa, 2005;
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, *Kompilatory. Reguły, metody i narzędzia*, Wydawnictwo Naukowo-Techniczne, seria *Klasyka informatyki*, Warszawa, 2002;
- Dora Giammarresi, Jean-Luc Pointy, Derick Wood, *A reexamination of the Glushkov and Thompson constructions*, Univeristà Ca'Foscari di Venezia, 1998.

RE	Champarnaud-Głuszkow
$\emptyset \in RE$	
$\varepsilon \in RE$	
$a \in \Sigma \Rightarrow a \in RE$	
	
$R_A, R_B \in RE \Rightarrow R_A R_B \in RE$	
$R_A \in RE \Rightarrow R_A^* \in RE$	

1.4 Dostarczony program

W plikach `z7b.y` i `z7b.1` znajduje się szkielet programu do tworzenia automatów na podstawie wyrażeń regularnych. Zawiera pełny analizator leksykalny i fragment analizatora składniowego. Należy w nim uzupełnić analizator składniowy. Wejściem programu jest tekst wyrażenia regularnego, gdzie $\Sigma = \{0, \dots, 9, a, \dots, z\}$. Wyjście jest plikiem w formacie programu `dot` z pakietu `graphviz` dostępnego pod adresem <http://www.graphviz.org/>. Na wyjściu są diagramy przejść stanów dla trzech automatów: automatu niedeterministycznego (wynik konstrukcji Champarnaud-Głuszkow), deterministycznego (automat Głuszkowa po determinizacji) i minimalnego automatu de-

terministycznego.

Uzupełnienia wymagają jedynie reguły składniowe umieszczone pomiędzy parami znaków %% w programie analizatora składniowego. Reguły zapisują, czym jest wyrażenie regularne (RE). Dostarczono jedynie regułę rozpoznającą symbol alfabetu lub pusty ciąg symboli ε . Należy dodać reguły dla pozostałych konstrukcji. Do **tworzenia stanu** służy funkcja `create_state()`. Nie ma parametrów. Wynikiem jest numer utworzonego stanu. Do **usuwania stanu** służy funkcja `delete_state()` z parametrem będącym numerem stanu do usunięcia.

Dla automatu odpowiadającego danemu (pod)wyrażeniu utrzymujemy listę jego stanów końcowych. Do **uczynienia** stanu **stanem końcowym** wykorzystujemy dwuparametrową funkcję `add_final()`. Odnosi ona końcowość stanu w tablicy `final` (`final[s] = 1` gdy stan jest stanem końcowym, 0 w przeciwnym wypadku) i dodaje stan do listy stanów końcowych występujących w danym podautomacie (automacie dla danego podwyrażenia). Pierwszym argumentem funkcji jest numer stanu, którym ma być końcowym, drugi – numer następnego stanu na liście stanów (-1 oznacza koniec listy). Funkcja zwraca numer stanu na liście stanów końcowych, tak aby można było podać w następnym wywołaniu tej funkcji dla tego samego podwyrażenia. Do **sprawdzenia końcowości stanu** służy funkcja `is_final()`. Jej parametrem jest numer stanu, zwraca 1, gdy stan jest końcowy. **Kończowość** stanu można **usunąć** za pomocą funkcji `delete_final()`. Funkcja nic nie zwraca, a argumentami są numer stanu i numer podwyrażenia/podautomatu (patrz funkcja `createRE()` dalej w tekście). Używana jest w obsłudze alternatywy. Funkcja `clear_finality()` **usuwa atrybut końcowości** dla wszystkich stanów z listy stanów końcowych. Parametrem jest indeks pierwszego stanu na liście (patrz funkcja `add_final()`). Wykorzystuje się ją w sklejaniu.

Dla listy stanów końcowych, `get_first_final()` podaje numer pierwszego stanu końcowego na liście, zaś `get_final_tail()` podaje dalszą część listy. Chcąc wykonać operację OPERACJA na wszystkich stanach końcowych pierwszego wyrażenia po prawej stronie reguły produkcji możemy więc napisać:

```
for (f = LAST($1); f != -1; f = get_final_tail(f)) {
    OPERACJA(get_first_final(f));
}
```

Do **łączenia list stanów końcowych** służy funkcja `merge_lists()`. Dodaje one drugą listę do końca pierwszej. Argumentami są indeksy pierwszych stanów list na liście stanów końcowych. Funkcja zwraca pierwszy argument (czyli połączoną listę).

W metodzie Champarnaud-Głuszkow często **łączy się pary stanów**. Operacja ta polega na kopiowaniu przejść drugiego stanu do pierwszego stanu. Wykonywana jest za pomocą funkcji `merge_states()`. Argumentami są numery dwóch stanów, które mają ulec połączeniu, wynikiem – numer pierwszego stanu z nowymi przejściami powstałymi w wyniku połączenia.

Ponieważ częstą operacją jest **dodawanie przejść jednego stanu do** stanów z całej listy stanów, to funkcja `merge_state_with_list()` wykorzystuje funkcje `merge_states()`, `get_first_final()` i `get_final_tail()` do wykonania takiej właśnie operacji. Pierwszym argumentem jest stan, drugim — lista stanów (końcowych). Kiedy łączone są zarówno stany początkowe, jak i listy stanów końcowych, wystarczy wywołać funkcję `merge_automata`. Dwoma argumentami tej funkcji są numery wyrażeń/automatów składowych, wynikiem — numer nowego wyrażenia, czyli indeks w tablicy `subREs` nowego automatu.

Do **tworzenia przejścia** służy funkcja `create_transition()`. Jej pierwszym parametrem jest numer stanu źródłowego, drugim – numer stanu docelowego, trzecim – etykieta przejścia. W metodzie Champarnaud-Głuszkow przejścia tworzone są tylko do obsługi pojedynczych symboli. W pozostałych przypadkach przejścia się kopiuje.

W celu składania większych wyrażeń z mniejszych musimy znać numery stanów początkowych i końcowych podwyrażeń. Aby uniknąć kłopotów z przekazywaniem struktur w czystym języku C, stan początkowy i lista stanów końcowych (patrz funkcja `add_final()`) zapisywane są w tablicy `subREs`. Dla bieżącego (właśnie tworzonego) wyrażenia należy do tego użyć funkcji `createRE()`. Wynikiem tej funkcji jest indeks elementu tablicy `subREs`. Należy go przekazać jako wynik konstrukcji składniowej w zmiennej \$\$ na końcu obsługi wyrażenia:

```
$$ = createRE(początkowy,końcowe); /* stan początkowy i lista końcowych bieżącego RE */
```

Do wyluskania stanu początkowego i listy stanów końcowych automatu odpowiadającego wyrażeniu służą makra `FIRST()` i `LAST()`, np. jeśli chcemy określić numer stanu początkowego automatu dla trzeciego wyrażenia w regule produkcji konstrukcji składniowej, piszemy:

```
FIRST($3) /* dla listy stanów końcowych wymieniamy „FIRST” na „LAST” */
```

1.5 Zadanie do wykonania

1. Dopisać obsługę wyrażenia pustego $\emptyset$ i wyrażenia w nawiasach.
2. Dopisać obsługę alternatywy. Choć operacja jest bardzo prosta (łączenie stanów początkowych i łączenie list stanów końcowych), trzeba pamiętać o możliwej końcowości stanów początkowych. Jeżeli stan q_{0B} był końcowy, to należy tę końcowość usunąć, a jeśli wtedy dodatkowo stan q_{0A} nie był końcowym, to należy go takim uczynić.
3. Dopisać obsługę sklejania. Wykorzystać priorytet operatora CONCAT (sklejenie to dwa wyrażenia po sobie; nie jest potrzebny dodatkowy operator). Przykład ustawiania priorytetu można znaleźć pisząc `info bison` (lub w emacsie `Ctrl-H i`, następnie `m bison` i klawisz `Enter`) i wybierając Examples/Infix Calc. W sklejaniu też należy zwrócić uwagę na końcowość stanów: jeśli $q_{0B} \in F_B$, to listę F_A należy dołączyć do listy F_B , w przeciwnym wypadku należy usunąć końcowość (`clear_finality()`) stanów z listy F_A .
4. Dopisać obsługę domknięcia przechodniego.
5. Dopisać rozszerzenie: operator domknięcia „+”. Tu należy także uzupełnić analizator leksykalny i ustalić priorytet.

Jeśli dla wyrażenia regularnego z przykładu powstają inne automaty niż podane na rysunkach, należy wypróbować działanie programu na najprostszych wyrażeniach typu „01” „0|1” „0*” i porównać wynik z rysunkami w tabelce. Jeśli jest OK, błędy mogą być w parametrach funkcji `createRE` – określeniu stanu początkowego i listy stanów końcowych. Automaty mogą mieć inną numerację stanów – wtedy wynik jest poprawny. Częsty błąd polega na użyciu zmiennej \$2 zamiast \$3, gdy \$2 oznacza „wartość” operatora.

1.6 Przykład

Wyrażenie: $0(0|1)^*0$ (ciąg zer i jedynek zaczynający się i kończący zerem).

Wynik w postaci tekstowej:

```
digraph "\"0(0|1)*0\"" {
    rankdir=LR;
    node[shape=circle];

    subgraph "clustern" {
        color=blue;
        n7 [shape=doublecircle];
        n [shape=plaintext, label=""]; // dummy state
        n -> n0; // arc to the start state from nowhere
        n0 -> n1 [label="0"];
        n3 -> n5 [label="1"];
        n3 -> n3 [label="0"];
        n5 -> n5 [label="1"];
        n5 -> n3 [label="0"];
        n3 -> n7 [label="0"];
        n5 -> n7 [label="0"];
        n1 -> n7 [label="0"];
        n1 -> n5 [label="1"];
        n1 -> n3 [label="0"];
        label="NFA"
    }

    subgraph "clusterd" {
        color=blue;
        d2 [shape=doublecircle];
        d [shape=plaintext, label=""]; // dummy state
        d -> d0; // arc to the start state from nowhere
    }
```

```

d0 -> d1 [label="0"];
d1 -> d2 [label="0"];
d1 -> d3 [label="1"];
d2 -> d2 [label="0"];
d2 -> d3 [label="1"];
d3 -> d2 [label="0"];
d3 -> d3 [label="1"];
label="DFA"
}

subgraph "clusterm" {
  color=blue;
  m0 [shape=doublecircle];
  m [shape=plaintext, label=""]; // dummy state
  m -> m1; // arc to the start state from nowhere
  m0 -> m0 [label="0"];
  m0 -> m2 [label="1"];
  m1 -> m2 [label="0"];
  m2 -> m0 [label="0"];
  m2 -> m2 [label="1"];
  label="min DFA"
}
}

```

Wynik w postaci diagramów:

